Neural Processing Letters (2023) 55:10655–10668
https://doi.org/10.1007/s11063-023-11343-9

On the Performance of new Higher Order Transformation Functions for Highly Efficient Dense Layers

Atharva Gundawar[1] · Srishti Lodha[1] · V. Vijayarajan[1] · Balaji Iyer[2,3] · V. B. Surya Prasath[2,3]

Accepted: 25 June 2023 / Published online: 23 July 2023
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2023

Abstract
Over the past few decades, a lot of new neural network architectures and deep learning (DL)-based models have been developed to tackle problems more efficiently, rapidly, and accurately. For classification problems, it is typical to utilize fully connected layers as the network head. These dense layers used in such architectures have always remained the same – they use a linear transformation function that is a sum of the product of output vectors with weight vectors, and a trainable linear bias. In this study, we explore a different mechanism for the computation of a neuron's output. By adding a new feature, involving a product of higher order output vectors with their respective weight vectors, we transform the conventional linear function to higher order functions, involving powers over two. We compare and analyze the results obtained from six different transformation functions in terms of training and validation accuracies, on a custom neural network architecture, and with two benchmark datasets for image classification (CIFAR-10 and CIFAR-100). While the dense layers perform better in all epochs with the new functions, the best performance is observed with a quadratic transformation function. Although the final accuracy achieved by the existing and new models remain the same, initial convergence to higher accuracies is always much faster in the proposed approach, thus significantly reducing the computational time and the computational resources required. This model can improve the performance of every DL architecture that uses a dense layer, with remarkably higher improvement in larger architectures that incorporate a very high number of parameters and output classes.

Keywords Dense layer · Neuron · Linear function · Quadratic function · Higher-order functions

1 Introduction

Deep learning (DL) models are routinely used to solve complex problems involving large datasets [1]. DL models based on artificial neural networks typically utilize regularization layers like dropout [2], pooling layers [3], Batch Normalization (BN), pruning [4], and quan-

Extended author information available on the last page of the article

tization [5] to mitigate the problem of exploding and vanishing gradients [6, 7]. DL models are data hungry due to their high model capacity and consequently require high training time. While these techniques were compatible with models which solved most problems, there were other techniques that involved ingenious methods like skip connections [8] to jump the gradients over multiple layers like InceptionNet [9, 10] or the idea of using blocks of convolution in VGG network (Visual Geometry Group). These models worked wonders on basic vision tasks. Nevertheless, they consumed a tremendous number of parameters, for example, the VGG-11 required a huge matrix of 25,088×4096, which required around 0.5 GB of RAM even at FP32 precision. This is where the idea of Network in Network was introduced. Starting from 2015, there has been an emergence of larger models that have specifically focused on addressing challenges related to object detection and tracking [11–13]. On the other hand, in NLP (or natural language processing), the problem of variable input was solved using RNN (Recurrent Neural Network). This field grew from simple recurrent models like GRU and LSTMs to models like Transformers and BERT, which have been the base for multiple seq-to-seq model implementations like google assistant, OpenGPT, and more. Right after RNN became popular to solve variable length input problems, ingenious developments like GRUs and LSTMs also came into existence. The developments did not stop there, and we now have huge progress in generative models and reinforced algorithms in all of these techniques. Nonetheless, one fundamental unit that stayed unchanged was the underlying transformation functions of the neurons in the networks.

As a neuron is the building block of any architecture, even with a small change in the transformation function of the neuron, a huge impact can be effected not only on the efficiency of the model but also, its model size and training time. This conventional/traditional mathematical function that maps the inputs of a neuron to its output can be explored and changes can be made to improve upon some features like convergence time and accuracy metrics. While the synaptic weights associated with a neuron serves as a repository of previous knowledge and memory, the somatic operations in the neuron integrate input signals over time and space. In the construction of artificial neural networks it is typical to simply compute a weighted sum of incoming inputs before applying nonlinear activation, we refer to such units as Ordinary Neural Units (ONUs). A natural extension is to consider higher order interactions between the features of the input signal. Equation (1) shows a second order neural unit where the original K dimensional input is transformed into a K^2 dimensional input by considering second order interactions.

$$z = f\left(\sum_{j=0}^{K}\sum_{i=0}^{K} x_i x_j w_{ij}\right) \quad (1)$$

Although higher order units can capture the correlation structure between features better than first order neurons, the number of terms combinatorically explode as the order increases. In this research, we build and test restricted higher-order transformation functions on a simple convolutional neural network, for the task of image classification. The term "dense layers" will be used to refer to "fully connected perceptron layers" throughout this study. To evaluate the results generated by the conventional neuron (as well as our newly proposed neurons), we used 2 benchmarked image classification datasets, namely, CIFAR-10 and CIFAR-100 [14]. On comparing the results of our proposed transformation function,

improved accuracies in both training and validation were observed on any fixed number of epochs meaning the convergence time was faster for these higher-order functions.

The remainder of the paper begins with a literature review of closely related research in Sect. 2. We highlight their major contributions and drawbacks. In Sect. 3, we discuss, in depth, the methodology adopted for the implementation of this study. Here, we go through every step of the research pipeline, beginning from the first sub-section (on defining custom layers), to the last (model evaluation). Then, we move to the results and analysis in Sect. 4 and shed light upon the several inferences that can be drawn from the work conducted for this research. This is followed by a conclusion of the study in Sect. 5, and finally, the references.

2 Literature Review

Minsky and Papert ([15]) were one of the earliest researchers to consider higher order perceptrons but considered them to be infeasible due to the exponential increase in the number of higher order terms. Giles ([16]) studied the learning, generalizability and invariance properties of higher order neural networks extensively. They designed higher order neural networks with translation invariant properties and experimented on the classic Exclusive-OR and TC problem. Softsky ([17]) explored correlations in high dimensions using Hebbian learning algorithm.

Naik et al. [18] considered the use of the honeybee mating optimization as an algorithm that governs the gradient decent of logits in Functional Link Neural Network (FLANN), the authors have proposed a new method for classification as well. The work has shown significant improvement compared with previous methods on numerical datasets with small input sequences. However, this model has been tested on datasets with small input sizes. In Mishra et al. [19], the authors have compared Multi-Layer Perceptron (MLP), FLANN and PSO-FLANN on a biomedical classification task. It was observed that MLP shows better results. Model for classifier fusion is also proposed. While the dynamic classifier fusion model resulted in a 3% increased accuracy, this technique is only limited to a single dataset. Research on applying the proposed models on other datasets within the scope of the targeted problem is missing, along with details about the architecture of the models used. Rout et al. [20] implement and compare different combinations of FLANN with models including Chebyshev, Lagurre, Legendre, and Power series. The testing of these models was limited to the simpler datasets that were popular prior to the advent of deep learning. The study conducted by Patra et al. [21] developed a FLANN model for the purpose of predicting the closing prices of stocks in the US market. A novel trigonometric FLANN approach was proposed, which shows improved results over traditional MLP networks. In all these papers, there is a lack of comparison with benchmarked datasets and models to validate the proposed methodologies. A new member to the FLANN family is introduced in Naik et al. [22] - the CRO-GDL-FLANN, which implements the traditional FLANN with gradient descent learning based on chemical reaction optimization. This model demonstrated notably improved performance compared to the other models examined in the study, however a comparative analysis with PSO-FLANN did not showcase statistically significant distinctions. Reddy and Varma [23] used machine learning models like radial basis functions support vector machines and compared them with a few neural network models such as MLP, FLANN,

Legendre Neural Network and Chebyshev Neural Network. Although a detailed comparison of these models was provided, it would have been helpful to include further information regarding the model architectures utilized and the training environment.

Lin et al. [24] proposed a modified ResNet architecture that incorporated a singular hidden neuron within each alternating dense layer, aiming to investigate the universal approximation property of these models. Despite constraining the number of hidden neurons in their proposed model, the researchers successfully attained comparable classification accuracy to that of a conventional ResNet model. Fan et al. [25] proposed an autoencoder architecture replacing the single-order dense layers with convolutional layers and quadratic dense functions. Although this paper achieved excellent results clinically on a targeted dataset, there is no information if the performance will be replicated on other datasets and architectures too.

In Kůrková et al. [26], a probabilistic approach to achieve a lower bound on errors in approximation (with a shallow signum perceptron) was proposed. It was observed that the total number of neurons (on finite domains) can help obtain this lower bound. However, the application of this proposal is restricted to particular domains. Moreover, the error approximation fails to give reliable results until a threshold of signum perceptron is present in a particular layer, in both binary- and real-valued functions. The authors in Gentile et al. [27] propose a new type of perceptron for on-line learning linear-threshold functions and suggest making this research a standard perceptron algorithm. The proposed algorithm converges faster than the known perceptron algorithms. The accuracy results of this algorithm are interesting as the model surpasses the conventional dense layer when tested with some datasets but performs equally bad with other datasets.

In Fan et al. [28], a new second order neuron was proposed to replace the original dense layer neuron. These models exhibited impressive performance in handling tasks with lower complexity, specifically in the context of fuzzy logic problems. However, the research did not delve into the specific workings of this principle when applied to more intricate neural networks. In their study, the authors additionally introduced a backpropagation algorithm with the aim of enhancing the backward pass process for weight updates in this novel second-order neural network. A difference in accuracies is observed when the models using conventional and new backpropagation techniques respectively are compared. Yet, these results are obtained only from benchmarked biomedical and engineering datasets, not sufficient to prove its working in real-world datasets. Some other relevant studies include Blondel [29] that presented new training algorithms and formulae for higher-order factorization machines which can be applied to neurons, but failed to fine-tune or explore any algorithm in depth, Du and Lee [30] which finds a relationship between the number of training samples and hidden nodes activated by a quadratic function but lacks research on how the local search algorithm using over parameterization discovers close global minimas, etc.

3 Methodology

In this study, the goal is to observe and analyze the change in accuracies by testing a simple architecture (consisting of convolution layers and dense layers), with neurons implementing different transformation functions. A comparison is done between 6 models, where the neurons in the first one uses the conventional linear transformation function to calculate the logits, while those in the other models use a higher order transformation function (up to the

degree of 6). The reason for terminating the analysis after the 6th degree function is discussed in the results section, along with a detailed discussion on the improved performance of the proposed model. The simple convolutional neural network used to test the proposed transformation functions is also presented.

An abbreviation used in this research to name the models is 'xn', where n is the degree of the transformation function of the model. For example, x6 is a model where the dense layer has a transformation function with the degree 6, and so on. In this section, the complete pipeline followed for implementation has been discussed. The details include sub-sections starting from data acquisition and custom layer definition, up to the evaluation of the trained model.

3.1 Defining Custom Layers

We start with the layer definition while implementing a custom layer. This step includes defining the data type of the variables that will be used directly as the coefficients or indirectly in the transformation function. In the next step, the layer is built by choosing the kernel type responsible for computing the logits in the forward pass. Finally, we define the transformation function which will compute the logits in the forward pass.

In the initialization section, the number of units and the default activation function that will augment the output logit are specified. These incorporate declaring and initializing the number and data type of units, along with other input independent variables. All the defined variables can be added to the scope of the parent class for easy access. Next comes the building section, mentioning the scope of the variables discussed above, and initializing the kernel for the computation of the logits. This section is called when the model is compiled, allowing the interpreter to allocate memory for each neuron accordingly. To define the call function of the layer called during the training of the model, the inputs taken by the layer are defined and passed to the transformation function, which returns the computed logit value to the training method. If an activation function is defined, the logit is passed through the activation function first, before returning the augmented logit value to the training method.

3.2 Data Overview

The datasets used for this research are CIFAR-10 and CIFAR-100. As mentioned beforehand, both of these are commonly used image classification datasets. CIFAR-10 and CIFAR-100 are labeled subsets of the "80 million tiny images" dataset. CIFAR-10 dataset contains 60,000 32×32 color images divided into ten classes, each with 6000 images. There are 50,000 training images and 10,000 test images. CIFAR-100 dataset, on the other hand, consists of 60,000 32×32 color images divided into 100 classes, each with 600 images. There are 50,000 training images and 10,000 test images. Both datasets are very well maintained by the choice of our deep learning framework: Tensorflow [31]. The Tensorflow datasets library has an inbuilt method to download the datasets which returns the data in a structured format. CIFAR-10 is about 163 MB and CIFAR-100 is about 161 MB in size.

3.3 Model Architecture And Model Compilation

The architecture used for this research, Fig. 1, is a sequential model, consisting of two sections. The first section consists of 3 convolutional layers, with the number of filters being 32, 64, and 64 from the first layer to the last in that order. Between every 2 consecutive convolutional layers, there is a MaxPooling layer, with a kernel size of 2×2. The input size of the first convolutional layer is dependent on the input size of the images, which in both cases (CIFAR-10 and CIFAR-100) is 32×32 with 3 channels (RGB).

This section of the model is connected with the next using a flatten layer, which flattens the output of the last convolutional layer into a 1-dimensional vector. The following section consists of either dense layers, or the custom-defined layers proposed in this study. There are 3 layers of each, and every layer is followed by a dropout layer with a dropout probability of 20%. Finally, a conventional dense layer is added as the classification head of the models. All layers have ReLU [32] as the activation function, except for the classification layer, which has a Softmax activation.

The above-described sequential CNN is built using the explained architecture. The model is compiled using the following two arguments. First, a pre-defined sparse categorical cross-entropy loss function with default settings, and second, a custom Adam optimizer [33] with gradient clipping activated [34] (clipvalue=0.5), a learning rate of 1e-05, and a beta_1 of 0.5. As this is a classification task, the choice of metrics is accuracy. An alternative to the sparse categorical cross-entropy function is the binary cross-entropy which is built specifically for 2 classes, but as the number of classes in the dataset is 2, both of these loss functions will return the same loss value. Hence, we do not choose this function.

A full higher order neural unit can be implemented as below:

$$Z_i(x) = f\left(W_o(i) + \sum_j W_1(i,j)x_j + \sum_j \sum_k W_2(i,j,k)x_j x_k + \cdots + \sum_j \cdots \sum_N W_N(i,j,\ldots N)x_{j\ldots}x_N\right) \quad (2)$$

Z_i is the output of the ith neuron, f is an activation function and N is the order of the neural unit and reflects the number of interacting terms considered in the design of the network. As the order increases, the number of higher order correlation terms increases exponentially. This also implies an exponential increase in the number of parameters of the model and results in models that massively overfit on real world data. We circumvent this combinatorial problem while retaining some of the useful properties of higher order networks by designing restricted higher order models. Specifically, we design networks that consider only the self-interacting terms from Eq. (2). Mathematically, such neural units can be represented as follows.

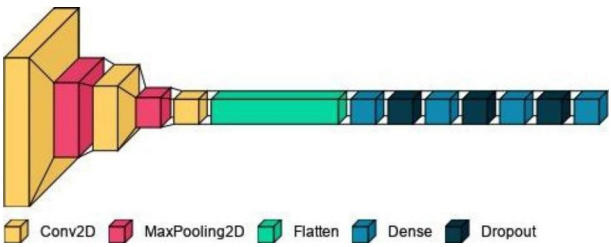

Fig. 1 Architecture of the CNN built

$$z = W_1^{(1)} x^n + W_2^{(1)} x \, (3)$$

where $x \in R^d, W_1^{(1)}, W_2^{(1)} \in R^{h \times d}$ are the trainable weight parameters. To get the hidden activation of the intermediate logit, we pass the logit or intermediate variable $z \in R^h$ throught the activation function ϕ.

$$h = \varphi \, (z) \, (4)$$

Now, we can obtain the output layer variable vector of length q. Assuming the output layer contains the weight vectors:$W^{(2)}1, W^{(2)}2 \in R^{q \times h}$,

$$o = W_1^{(2)} h^n + W_2^{(2)} h \, (5)$$

We can denote the loss **L** as the value returned by the above pre-defined loss function to get an output **h**. This value combined with the real target label **y** will be used to calculate the loss for a single example, returning the loss **L**,

$$L = l \, (o, y) \, (6)$$

Considering the hyperparameter term λ, we can include l2 regularization (s), and can calculate it as:

$$s = \frac{\lambda}{2} \left(\left| (W_1^{(1)}) \right|_F^2 + |W_2^{(2)}|_F^2 \right) (7)$$

Here, λ denotes the regularization weight, typically set to 1. For restricted second-order neurons, the self-interacting terms reflect the power in the signal. We can compute the gradient of the neuron with respect to W_1 as follows.

$$\nabla {W_1}^{(1)} = x^{n\prime} \, (8)$$

During training, there is a risk of exploding and vanishing gradient if the power in the signal is too low or too high. To mitigate this, we regularize our models using dropout, L2 regularization and gradient clipping. We note that using only the higher order x^n can result in vanishing gradients when the values are very small. By using a balanced formulation (3) we avoid this issue. Experimentally we tested models of orders higher than 2 however found that models of order 2 work best.

3.4 Preparing Custom Training loop, Model Training And Evaluation

There were some changes or augmentations made to the default training steps for recording the results in a more concise manner. This was also to avoid the exploding gradient problem during the training of x3, x4, x5 and x6. To record epoch wise training and validation accuracies and losses, callback functions were implemented. To handle change in the learning rates and avoid the exploding gradient problem in the initial epochs, learnings from momentum were used to increase the learning rate after 10 every 10 epochs. During the training of

Table 1 Training accuracy obtained on varying epochs for 2 datasets

Dataset	Number of epochs	Conventional accuracy (%)	New training accuracies (%)				
			x2	x3	x4	x5	x6
CIFAR-10	1	26.62	33.89	27.29	24.30	21.41	30.20
	2	42.76	50.56	45.86	44.98	41.97	48.55
	5	54.79	62.31	58.96	58.22	57.17	60.99
	10	61.92	69.85	66	64.33	62.44	65.18
	20	69.82	76.69	71.93	62.47	63.76	66.94
CIFAR-100	1	2.06	1.95	2.04	1.91	1.98	2.07
	10	10.46	15.34	13.06	12.01	12.03	11.66
	20	15.90	22.65	21	19.28	19.16	17.92
	30	19.58	27.78	26.13	23.08	22.95	21.87
	40	22.75	31.70	30.48	25.66	26.59	25.30
	50	25.35	35.03	33.55	28.23	28.85	27.61

Table 2 Validation accuracy obtained on varying epochs for 2 datasets

Dataset	Number of epochs	Conventional accuracy (%)	New validation accuracies (%)				
			x2	x3	x4	x5	x6
CIFAR-10	1	41.87	46.77	42.93	40.37	38.93	45.34
	2	49.98	55.10	51.41	52.44	49.60	49.29
	5	57	62.67	61.53	60.71	57.70	60.38
	10	62.92	66.52	63.12	61.72	60.69	64.36
	20	67.18	68.20	65.87	60	57.36	63.60
CIFAR-100	1	2.54	3.47	2.58	2.29	2.88	2.63
	10	14.70	21.04	15.06	14.40	11.28	13.60
	20	20.90	27.08	22.50	21.23	21.31	19.07
	30	23.76	30.74	27.09	24.81	23.89	23.35
	40	25.76	33.75	30.38	26.76	27.36	25.48
	50	28.83	35	30.43	28.05	28.56	26.98

the model, TensorBoard was used to monitor the losses and the accuracies, visualized live on the dashboard.

4 Results and Analysis

As already mentioned, for analyzing the proposed methodology, we tested the models on CIFAR-10 and CIFAR-100. Both training and validation accuracies were recorded (refer Tables 1 and 2), for six cases (starting from the conventional linear transformation function, up to the higher order function of 6th degree). Due to limited computational resource, the training and validation was conducted up to 20 epochs on CIFAR-10 and 50 epochs on CIFAR-100. Additionally, epoch-wise training time was on an average 20,000 milliseconds, so the training of the 6 models on CIFAR-100, for instance, took 6*20,000*50 milliseconds (over 1.5 h). The training was done on a Tesla T4 GPU with memory size of approximately 16GB. These results have also been visually represented in Figs. 2 and 3 respectively. We notice that the quadratic higher order neuron-based architectures tend to perform better overall. It must be noted that the aim of our study is not to improve model accuracies for a

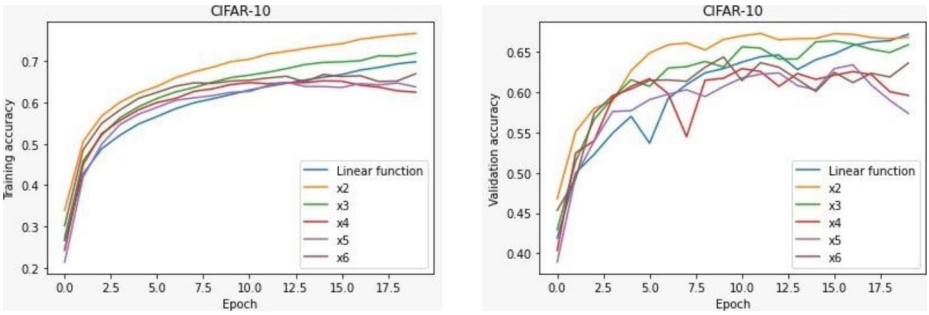

Fig. 2 **(a)***Left* Training and **(b)***Right* validation accuracies plotted for the first 20 epochs (CIFAR-10)

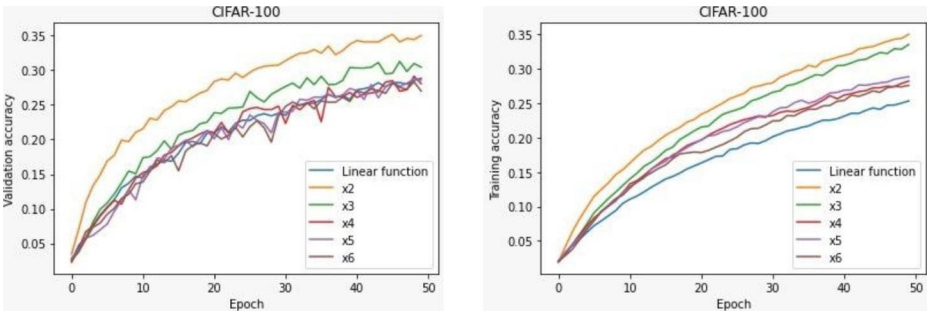

Fig. 3 **(a)***Left* Training and **(b)***Right* validation accuracies plotted for the first 50 epochs (CIFAR-100)

particular architecture or dataset, but to propose an all-together novel approach to achieve much better accuracies than conventional models on any epoch. Hence, although the training process is terminated before a decent accuracy is attained, the goal of this study is clearly demonstrated within the training conducted, and there was no requirement to train the models any further. Nevertheless, datasets like CIFAR-100 require hundreds of epochs to train, even with big architectures like InceptionNet [35] and EfficientNetV2 [36]. When our higher order transformation-based models are run for higher epochs, we observed the training accuracy stagnated while the improvement in validation accuracies were marginal.

A small observation can be made about the change in model weight, which is affected with the new transformation functions. While a linear dense layer model had a total of 201,156 trainable parameters, the architecture with these higher order transformation functions had 70% more parameters (342,468), which remained consistent through all the powers. Moreover, since no layer or logit in the architecture was frozen (or untrainable), the total number of parameters and the trainable parameters are equal. The increase in the number of parameters does not affect the models overall training time.

The methodology adopted in this research drastically reduces the number of epochs needed to arrive at a particular accuracy. This is why the overall computational time and costs required for training and validation of any model is highly reduced. This inference can be drawn from the results presented in Tables 1 and 2. For instance, if we consider the training time on the CIFAR-100 dataset, we observe that the conventional model with linear neurons take almost 40 epochs to attain an accuracy of approximately 22.70%, while the

new model with quadratic transformation function (x2) takes only 20 epochs. Similarly, on CIFAR-10, the conventional accuracy hits 69.82% at the 20th epoch, while the same results are achieved by x2 in half the time again (10 epochs). Correspondingly, the improvement is maintained through models' validation. For example, 67% accuracy during 20th and 10th epochs for linear model and x2 respectively on CIFAR-100, 25.76% accuracy in 40 epochs and less than 20 epochs on CIFAR-100, and so on. Clearly, the computational time and resource requirement is reduced to less than half. We can extend this observation to estimate that if a heavy deep learning architecture with conventional neurons can reach the accuracy of 92% in 50 epochs, the same model will produce a 92% accuracy in much less than 30 epochs with the new neurons. This hypothesis can be successfully validated by implementing other layers, like Convolutional and Recurrent layers. Nevertheless, both the types of neurons and dense layers might ultimately reach the same accuracy, just with a drastically different final number of epochs. We remark that by using higher order transformations which increases the number of parameters naturally the training convergence is faster. Similar increase in parameter complexity of traditional linear neurons can easily overfit.

We note that the model with quadratic transformation function always performed the best on both the datasets and for almost every epoch. The added dimension to the equation of a neuron does not affect the training time much. Moreover, it can be inferred the performance is not directly proportional to the degree of the transformation function used. In fact, with higher epochs, the powers become useless. That is, any added dimension after power 3 does not benefit or improve the model at all, because of the exponential change in the value of the gradients. In other words, the extra dimension is rendered useless, because when we deal with such high powers, the following two problems arise. First, when the higher order terms are small the gradient is very small, its contribution to the gradient is negligible. However, the presence of the linear weight vector and bias term ensures that the gradient does not vanish. On the other hand, if the higher order terms are very big, the exponential power makes the value of the term equal to the maximum tensor value and dominates the other 2 terms (the liner weight vector and the bias) since adding any value to the maximum tensor value will not change the result. We mitigate the risk of exploding gradients by clipping the gradient value to a maximum threshold, ensuring the models can be trained without this problem.

In our research, we experimented with models consisting of higher order transformations up to degree 6. All higher order DL models beyond x3 (x4, x5 and x6) started to consistently yield lower accuracies in the CIFAR-10 training after the 13th epoch. Even though the linear transformation function was performing worse than all the other transformation functions in the first few epochs, after the 9th epoch, the accuracies of the models beyond x3 flatline. That is, there was no noticeable improvement in the accuracy. Even in the case of CIFAR-100, we can see that x2 and x3 performed better than all the other models. Due to all the reasons discussed before, as well as the results obtained here, it is safe to conclude that this trend will continue, and the performance will not improve for models with degrees beyond 6.

With the goal of testing the higher order framework on more complex models we ran transfer learning experiments using the higher order dense layer and contrast it with linear order dense layer. For these experiments we restricted to quadratic dense layers and CIFAR-10 dataset. We consider the ResNet50 and MobileNetV2 architectures and show the results on CIFAR-10 dataset in Table 3. During training, we freeze the weights of these models and extract the feature layer from these models. The only trainable parameters are the ones in

Table 3 Validation accuracy of ResNet50, and MobiletNetV2 on the CIFAR-10 dataset with linear and quadratic neurons-based models

Model	Dense Layer	Accuracy
ResNet50	Linear	0.64
	Quadratic	0.66
MobileNetV2	Linear	0.54
	Quadratic	0.52

the dense layers. We trained both models for 50 epochs with an initial learning rate of 3e-3, the batch size was set to 256 and the gradient clipping value was set to 2. We used 64 hidden units in the dense layer for both the quadratic and linear case. For ResNet50, we obtain a test accuracy of 0.64 on CIFAR-10 for the linear dense layer and 0.66 for the quadratic dense layer indicating a slight improvement. In the case of MobilenetV2, we obtain a test accuracy of 0.54 for the linear layer and 0.52 for the quadratic dense layer. Finally, we contrast these results with a second order dense layer terms. In both cases we notice a significant drop in performance.

To test how the new transformation function compares to an architecture built with conventional dense layers, when both have the same number of parameters, we extended the architecture in Fig. 1 to accommodate more dense layers and make the model deeper. To achieve the same results while testing models with the linear and higher-order functions, we kept increasing the number of dense layers in the model from 3, until 15, where the linear model had the same number of parameters as the other models with 3 layers. This, as expected, caused the vanishing gradient problem in the linear model, and rendered the training useless. The training and validation accuracies remained negligibly small, and there were no significant changes in the losses throughout the training of 50 epochs. This clearly implies two things. First, the model with a higher order transformation function in the dense layers performs better with the same architecture as a linear model. Moreover, we can overcome the vanishing gradient problem in cases where it is quite active (in deeper architectures), by reducing the number of dense layers and converting the conventional transformation function to the researched higher order transformation function. Hence, this approach helps tackle a very common issue encountered in the deep learning area. Finally, we remark that dense layers with recurrence and self-attention-based combinations can also be considered wherein such higher order transformations can be taken into account.

5 Conclusion

A novel approach to significantly reduce the computational time and resources required for the training of deep learning (DL) architectures using dense layers has been proposed in this study. By targeting the smallest unit that builds a dense layer, a perceptron, a universal method for improvement has been successfully implemented. This is done by experimenting six different transformation functions to replace the conventional function that a neuron uses while computing its output (sum of a trainable bias and products of logit and weight vectors). All these new transformation functions convert the linear sum to a higher order function (quadratic, cubic, and so on), by adding a new term to the equation. To the best of our knowledge, such an approach has not been followed in any study so far. While some studies explore the idea of a different transformation function (discussed under the literature

review section), most of them only test the performance of quadratic functions, and in limited areas. When the proposed methodology is tested on two popular image classification datasets (CIFAR-10 and CIFAR-100), the predicted improvement is experimentally proven correct. All the results have been included in the study. While all the new models perform better, the model with quadratic transformation function outperforms all the others. In fact, it achieves any particular accuracy in half (or less than half) the number of epochs than a linear model. In the analysis section, it is explained why the second-order equation is optimal, and the added dimensions are not very fruitful (due to exponential change in gradient values). A change in the weight of the new models is also observed. Among the many significant inferences, it is important to note that our model solves another issue normally encountered in DL domain, namely the vanishing gradients problem. All of these have been discussed in depth in the analysis section. Since these models were only tested on a small custom-built architecture, we can predict similar (or better) improvement for any dense layer-based deep learning architecture, with larger ones yielding even better results (due to higher number of parameters). Lastly, this method can also be tested further with yet different transformation functions, such as with those involving exponents.

Author Contributions A.G. and S.L. conceived and designed the models. A.G., S.L., V.V., B.I., V.B.S.P. discussed the methodology and experimental analysis. V.V. and V.B.S.P. supervised the work. A.G. S.L. V.V. B.I. V.B.S.P. wrote the manuscript. All authors reviewed the manuscript.

Declarations

Conflict of Interest The authors declare that they have no conflict of interest.

References

1. LeCun Y, Bengio Y, Hinton G (2015) “Deep Learn " Nat 521(7553):436–444
2. Srivastava N, Hinton G, Krizhevsky A, Sutskever I, Salakhutdinov R (2014) Dropout: a simple way to prevent neural networks from overfitting. J Mach Learn Res 15(1):1929–1958
3. Lin H, Jegelka S (2018) Resnet with one-neuron hidden layers is a universal approximator. Adv Neural Inf Process Syst, 31
4. Vadera S, Ameen S (2022) Methods for pruning deep neural networks. IEEE Access 10:63280–63300
5. Gholamalinezhad H, Khosravi H (2020) Pooling methods in deep neural networks, a review. arXiv preprint arXiv:2009.07485.
6. Yang J, Shen X, Xing J, Tian X, Li H, Deng B, Hua XS (2019) Quantization networks. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (pp. 7308–7316)
7. Bengio Y, Simard P, Frasconi P (1994) Learning long-term dependencies with gradient descent is difficult. IEEE Trans Neural Networks 5(2):157–166
8. Pascanu R, Mikolov T, Bengio Y (2013) On the difficulty of training recurrent neural networks. In International conference on machine learning (pp. 1310–1318). PMLR
9. Orhan AE, Pitkow X (2017) Skip connections eliminate singularities. arXiv preprint arXiv :1701.09175
10. Szegedy C, Liu W, Jia Y, Sermanet P, Reed S, Anguelov D, Rabinovich A (2015) Going deeper with convolutions. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 1–9)
11. Fan F, Cong W, Wang G (2018) Generalized backpropagation algorithm for training second-order neural networks. Int J Numer Methods Biomed Eng, 34(5), e2956
12. Tan M, Le Q (2021) Efficientnetv2: Smaller models and faster training. In International Conference on Machine Learning (pp. 10096–10106). PMLR
13. Lan W, Dang J, Wang Y, Wang S (2018) Pedestrian detection based on YOLO network model. In 2018 IEEE international conference on mechatronics and automation (ICMA) (pp. 1547–1551). IEEE
14. CIFAR dataset. https://www.cs.toronto.edu/~kriz/cifar.html [Accessed 7 June 2023]

15. Minsky ML, Seymour A, Papert (1988) "Perceptrons: expanded edition."
16. Giles C, Lee, Maxwell T (1987) Learning, invariance, and generalization in high-order neural networks. Appl Opt 26(23):4972–4978
17. Softky WR, Daniel M (1991) Kammen. "Correlations in high dimensional or asymmetric data sets: hebbian neuronal processing. Neural Netw 4(3):337–347
18. Naik B, Nayak J, Behera HS (2015) A honey bee mating optimization based gradient descent learning–FLANN (HBMO-GDL-FLANN) for Classification. In Emerging ICT for Bridging the Future-Proceedings of the 49th Annual Convention of the Computer Society of India CSI Volume 2 (pp. 211–220). Springer, Cham
19. Mishra S, Shaw K, Mishra D, Patnaik S (2012) N enhanced classifier fusion model for classifying biomedical data. Int J Comput Vis Rob 3(1–2):129–137
20. Bebarta DK, Rout AK, Biswal B, Dash PK (2012) Forecasting and classification of Indian stocks using different polynomial functional link artificial neural networks. In 2012 Annual IEEE India Conference (INDICON) (pp. 178–182). IEEE
21. Patra JC, Thanh NC, Meher PK (2009) Computationally efficient FLANN-based intelligent stock price prediction system. In 2009 International joint conference on neural networks (pp. 2431–2438). IEEE
22. Naik B, Nayak J, Behera HS (2016) An efficient FLANN model with CRO-based gradient descent learning for classification. Int J Bus Inform Syst 21(1):73–116
23. Reddy ADG, Varma LT (2014) Wind power forecasting without using historical data. In International conference on advances in electrical engineering (ICAEE) (pp. 1–3).d
24. Lin H, Jegelka S (2018) Resnet with one-neuron hidden layers is a universal approximator. Adv Neural Inf Process Syst, 31
25. Fan F, Shan H, Kalra MK, Singh R, Qian G, Getzin M, Wang G (2019) Quadratic autoencoder (Q-AE) for low-dose CT denoising. IEEE Trans Med Imaging 39(6):2035–2050
26. Kůrková V, Sanguineti M (2017) Probabilistic lower bounds for approximation by shallow perceptron networks. Neural Netw 91:34–41
27. Gentile C, Vitale F, Brotto C (2007) On higher-order perceptron algorithms. Adv Neural Inf Process Syst, 20
28. Fan F, Cong W, Wang G (2018) A new type of neuron for machine learning. Int J Numer Methods Biomed Eng, 34(2), e2920
29. Blondel M, Fujino A, Ueda N, Ishihata M (2016) Higher-order factorization machines. Adv Neural Inf Process Syst, 29
30. Du S, Lee J (2018) On the power of over-parametrization in neural networks with quadratic activation. In International conference on machine learning (pp. 1329–1338). PMLR
31. Abadi M (2016) TensorFlow: learning functions at scale. In Proceedings of the 21st ACM SIGPLAN International Conference on Functional Programming (pp. 1–1)
32. Agarap AF (2018) Deep learning using rectified linear units (relu). arXiv preprint arXiv:1803.08375
33. Kingma DP, Ba J (2014) Adam: A method for stochastic optimization. arXiv preprint arXiv:1412.6980.
34. Zhang J, He T, Sra S, Jadbabaie A (2019) Why gradient clipping accelerates training: A theoretical justification for adaptivity. arXiv preprint arXiv:1905.11881
35. Szegedy C, Vanhoucke V, Ioffe S, Shlens J, Wojna Z (2016) Rethinking the inception architecture for computer vision. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 2818–2826)
36. Tan M, Le Q (2019) Efficientnet: rethinking model scaling for convolutional neural networks. In: International conference on machine learning, vol 97

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Authors and Affiliations

Atharva Gundawar[1] · Srishti Lodha[1] · V. Vijayarajan[1] · Balaji Iyer[2,3] · V. B. Surya Prasath[2,3]

✉ V. B. Surya Prasath
prasatsa@uc.edu; surya.iit@gmail.com

Atharva Gundawar
atharva.n.gundawar@gmail.com

Srishti Lodha
srishti2k1@gmail.com

V. Vijayarajan
vijayarajan.v@vit.ac.in

Balaji Iyer
iyerbs@mail.uc.edu

1 School of Computer Science and Engineering, Vellore Institute of Technology Vellore, Vellore, Tamil Nadu, India

2 Division of Biomedical Informatics, Cincinnati Children's Hospital Medical Center, Cincinnati, OH 45229, USA

3 Departments of Pediatrics, Biomedical Informatics, Computer Science, University of Cincinnati, Cincinnati, USA